

שנקר ביי"ס גבוה להנדסה ולעישוב
החוג להנדסת תוכנה
קורס 'רשתות תקשורת' (קורס מספר 3503836)
תשפ"ו – סמסטר ב'

מרצה: יצחק נודלר

תרגיל בית מס' 4 – פתרון

בהרצאות הצגנו את "ארכיטקטורת שלושת הרבדים".
רובד א': הרובד הקונספטואלי
רובד ב': רובד ממשק התוכנה וההפשטה בארכיטקטורת ה-TCP.
רובד ג': רובד הליבה וניהול המשאבים הפיזיים

בשאלות להלן, המשמעות של המונחים 'רובד א' ו- 'רובד ב' היא בהתאם לאמור לעיל.

שאלה מס' 1

במסמך ה-RFC של TCP מתואר כי TCP הוא מוכוון-חיבור (Connection-Oriented).
א. מדוע המאפיין הזה הוא תשתית חובה (Prerequisite) למימוש RDT?
ב. האם ייתכן פרוטוקול אמין שאינו מוכוון חיבור?

העברת נתונים אמינה (RDT) דורשת ניהול מצב 'אגרסיבי': מעקב אחר מספרי רצף, חוצצי זיכרון לשידור חוזר, ושעוני עצר (Timers). כל אלו דורשים זיכרון לוגי מתמשך שמוקצה בהתחלה ומנוהל לאורך החיבור – משאבים אלו מוחזקים בתוך ה-TCB שנוצר רק עם הקמת ה"חיבור". לכן, ללא הקמת ה"חיבור" (מצב לוגי משותף), הקרנל לא יכול לעקוב אחר אובדן נתונים. פרוטוקול אמין שאינו מוכוון חיבור (כמו UDP עם תוספת אמינות) יצטרך לנהל את ה-State הלוגי בדרך אחרת (למשל, בכל מנה בנפרד), אך ב-TCP הדרך ל-RDT עוברת דרך הקצאת משאבים מקדימה.

שאלה מס' 2

א. האם ה-Kernel של TCP מתייחס לכל קריאה ל-send של האפליקציה כאל כזו שמעבירה לו הודעה של פרוטוקול שכבה 5?
ב. מהי ההשלכה של כך על האופן שבו האפליקציה המקבלת צריכה להתמודד עם הנתונים?
*TCP מנהל את הנתונים כזרם בתים ולא כהודעות.
הקרנל יאורז בתים ל-Segments בהתאם ל-MSS.
האפליקציה בצד המקבל חייבת לדעת שהנתונים לא יגיעו בהכרח בגושים ששלחה האפליקציה השולחת, ולכן היא צריכה להפעיל לוגיקה משלה (למשל, סימני הפרדה) כדי לזהות הודעות בתוך ה-byte stream.*

שאלה מס' 3

- א. הסבירו את המנגנון ההנדסי המאפשר ל-Receiver "לפרסם" את קיבולת הזיכרון שלו.
ב. מה יקרה לשולח במידה וחלון הקבלה הוא אפס?

המקבל משתמש בשדה $rwnd$ בכותרת ה-TCP כדי לדווח על מקום פנוי בחוצץ הקליטה (Receive Buffer).
אם הערך הוא 0 (Zero Window State), השולח מחויב לעצור את השליחה כדי למנוע אובדן מידע כתוצאה מהצפת החוצץ של המקבל.

שאלה מס' 4

כיצד Delayed ACK משפיע על מודל ה-RDT התיאורטי?

ה-ACK ברוב א' [= הרובד הקונספטואלי] הוא אישור קבלה (Acknowledgement) המבטיח אמינות.
בהקשר של המימוש [= ברובד ב'], הוא כלי עבודה לניהול זרימה.
ה-Delayed ACK הוא אופטימיזציה שבה הקרנל "מעכב" את השליחה כדי לנסות לעשות Piggybacking עם מידע היוצא לכיוון ההפוך.
מבחינת רובד א', זה נראה כעיכוב בתקשורת, אך זהו tradeoff הנדסי שמגדיל את התפוקה (Throughput) על חשבון תגובתיות (Latency) מיידית.

שאלה מס' 5

האם ה-MSS הוא מגבלה תיאורטית או הנדסית? נמקו.

ה-MSS הוא הגדרה הנדסית שנובעת ממגבלת ה-MTU של שכבת הערוץ (Link Layer) כדי למנוע פרגמנטציה בנתבים/מתגים.
ברובד א', אנחנו מדברים על "זרם בתים", אך ברובד ב' אנחנו צריכים לחתוך את הזרם ליחידות שכרטיס הרשת יודע להעביר ללא בעיות.
ה-TCB שומר את ערך ה-MSS שסוכם במהלך ה-Handshake כדי שהפונקציה שכותבת ל-Socket תדע מתי "לחתוך" את המנה הבאה.

שאלה מס' 5

האם החזרה של הפונקציה `send()` משמעותה אישור כי הבתים הועברו לנמען [= פרוטוקול שכבה 5] באפליקציה?

ממש לא !

שאלה מס' 6

TCP ממיצג רשומת socket על ידי רשומת סוקט. או : TCB [= TCP Control Block] אילו שדות ברשומה זו חייבים להיגזר מרובד א', ואילו מרובד ב'?

מרובד א': *State* (מכונת המצבים), *Sequence Numbers* (עבור ה-RDT), *ACK Number*. אלו הגדרות שבלעדיהן הפרוטוקול אינו TCP.

מרובד ב': *Flow Control variables*, *Timer structures*, *Receive/Send Buffer pointers* (כמו *rwnd* גולמי), *Interface pointers*. אלו שדות מימוש שנועדו להריץ את התיאוריה על חומרה ספציפית.

שאלה מס' 7

האם באג במימוש הופך את הפרוטוקול ל"לא-TCP"?

ה-TCP מוגדר ע"י המימוש הסטנדרטי שלו.
אם מימוש קרנל מסוים מבצע אופטימיזציה שגויה, הוא עלול לא לתקשר עם מימושים אחרים (Interoperability).
ברובד א', התיאוריה מושלמת, אך ברובד ב' הפרוטוקול מוגדר "דה-פקטו" על ידי המימושים הגדולים (כמו בלינוקס).

שאלה מס' 8

האם גודל ה- Receive Buffer (רובד ב') מגביל את ה- Window Size (רובד א')? הסבירו את התלות.

כן.

גודל ה- Receive Window (רובד א') המפורסם ללקוח לא יכול לעלות על גודל החוצץ הפיזי ב- MM שהוקצה בתוך ה- TCB (רובד ב').
אם הזיכרון נגמר, חלון הקבלה חייב להצטמצם.
רובד א' מגדיר את גבול התיאוריה, ורובד ב' הוא האילוץ הפיזי שקובע את גודל החלון בפועל.

שאלה מס' 9

מדוע הקרנל בוחר לשמור סגמנטים ב- sk_sndbuf גם לאחר שהם נשלחו בפועל על גבי הכבל? הסבר את הקשר בין פעולה זו למודל ה- RDT.

ברובד הקונספטואלי, TCP חייב להבטיח אמינות.
כיוון שהרשת היא "Best Effort" ועלולה לאבד מנות, הקרנל חייב להחזיק עותק של הנתונים שטרם אושרו (Unacknowledged).
הם "בני ערובה" בחוצץ משום שאם לא יגיע ACK עבורם בזמן, ה- TCP יזדקק להם כדי לבצע שידור חוזר (Retransmission).

שאלה מס' 10

אם ה- MSS בחיבור הוא 1460, מדוע האפליקציה ייכולה' [= 'מותר לה'] לבקש לשלוח 2000 בתים? כיצד הקרנל יטפל בזה מבחינת ה- Segmentation?

הקרנל יבצע חלוקה ל- Segments.
סגמנט אחד יהיה בגודל 1460 (ה- MSS) והסגמנט השני יהיה בגודל 540 (היתרה).
TCP מנהל את הפיצול הזה כדי להתאים את המידע ל- MTU של הרשת.

שאלה מס' 11

הסבירו את הקשר בין Delayed ACK לבין Piggybacking של נתונים מהשרת חזרה ללקוח.

השרת ממתין זמן קצר לפני שליחת ה-ACK.
אם בזמן זה האפליקציה בשרת צריכה לשלוח נתונים, השרת "רוכב" על חבילת הנתונים הזו ומכניס לתוכה את ה-ACK.
זהו חיסכון כפול: פחות Header (חיסכון ברוחב פס) ופחות Interrupts למעבד (פחות פסיקות).

שאלה מס' 12

אם ה- sk_sndbuf מלא, האם האפליקציה תצליח להמשיך לשלוח נתונים?

לא.
ה- Kernel יחסום את האפליקציה עד שיתפנה מקום ב- Buffer (כלומר עד שיגיעו ACKs מהצד השני).
זו הדרך שבה TCP מבצע Back-pressure על האפליקציה.

שאלה מס' 13

מדוע ה- ISN נשמר ב- TCB לאורך כל חיי החיבור?

ה- ISN הוא נקודת האפס (Reference Point).
כל חישוב של Sequence Numbers עתידיים נעשה יחסית אליו.
אם נאבד אותו, אזי TCP לא ידע להשוות את מספרי הרצף הנכנסים לנתונים הקיימים בחוצץ.